

Java Sheet Notes

class - collection of obj., memory x
obj - real world entity, memory ✓

* Java

↳ high-level, object oriented programming lang. designed to be portable, secure & easy to use.

- develop → Sun Micro system → 1995

characteristics.

- Simple & easy to learn
- Object-oriented.
- Platform-independent
- Secure
- Robust - less chance to crash - why?
• Strong memory mgt.
• No pointer variable → containers, store value.
- Multithreaded.

Flavours of Java

- ↳ Java SE (desktop/server app)
- ↳ Java EE (web based app)
- ↳ Java ME (mobile phones)

* Java vs C++

Java

- Platform-independent
- No pointers
- Not supported multiple inheritance
- Highly secure

C++

- platform-dependent.
- Support pointers.
- Supported directly.
- Less secure.

Data Type

type of data

Variable can hold

- Primitive (built-in)
- Non primitive (Referenced)

* operators → perform specific opⁿ.

- ↳ Arithmetic → (+, -, *, /, %)
- ↳ Assignment → (=, +=, -=, *=, /=, %=)
- ↳ Relational → (==, !=, >, <, >=, <=)
- ↳ Logical → (&&, !)
- ↳ Unary → (+, -, ++, --, !)
- ↳ Bitwise → (&, ^, ~, <<, >>)

* JVM, JDK, JRE

- JVM → Java Virtual Machine → executes java bytecode
- JDK → Java Development Kit → JRE + dev. tools
↳ req. for developing app.
- JRE → JVM + libraries + other runtime tools.
↳ provides everything needed to run java programs.

* Types of Variable

- ↳ local → inside method
- ↳ Instance → inside class but outside method, belong - obj.
- ↳ Static → Belongs to class.

* Control Statements.

↳ Controls - flow of execution.

- ① Decision Making → (if, else, else if, switch)
- ② Looping st. → (for, while, do while)
- ③ Jump st. → (break, continue, return)

* Primitive (8)

↳ byte, short, int, long, float, double, char, boolean

* Non primitive

↳ String, Arrays, classes, interfaces

5 ways to create obj:-

- ① New keyword
- ② new Instance() method
- ③ clone()
- ④ deserialization
- ⑤ factory method

* Classes & Objects.

- class → blueprint for creating obj.
- obj → real instance of class | physical entity.
- field → var inside class.
- Method → Functⁿ → inside class
- Instantiatⁿ → creating obj using new keyword.

* Constructor

- ↳ block of code used to initialize obj.
- name as same as class name
- does not have return type.

Syntax:-

```
class One {  
    constructor One() {  
    }  
}
```

Types.

- ↳ Default → no param.
- ↳ Parameterized → with param.

* Method vs Constructor

Method

- any name
- has return type
- called manually
- def. behavior

Constructor

- Must match class name
- does not have return type
- called automatically.
- initialize obj.

Packages =
classes +
interfaces

* Constructor chaining

- ↳ calling one constructor from another within same class.
- helps reuse initializatⁿ code, avoid duplicatⁿ.

Types

- ↳ within same class → this()
- ↳ from parent class → super()

* OOP

↳ Object-oriented programming

- paradigm, organizes slw design around objects.

Goals

- ↳ Reusability
- ↳ Modularity
- ↳ Maintainability
- ↳ Extensibility.

Pillars (4)

- ↳ Inheritance
- ↳ Polymorphism
- ↳ Encapsulatⁿ
- ↳ Abstractⁿ

Constructor overloading



multiple constructors
in one class with
diff. param. list.

Copy Constructor



creates new obj. by
copying fields from
another existing obj.

→ Java does not provide
built-in copy constructor.

Deep Copy vs Shallow Copy

• Deep Copy

↳ copies ref.

• Shallow Copy

↳ copies everything

- **Inheritance** → parent-child relationship
 ↳ class acquires properties & behaviour of another class.
 → use 'extends'

eg. class Parent {

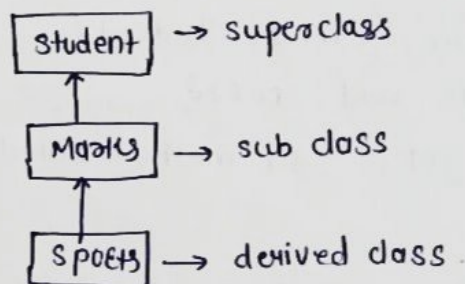
 }
 class child extends Parent {
 }

Types

- Single →
- Multilevel
- Hierarchical
- Hybrid
- Multiple.

- Single → subclass derived from one super class.

- Multilevel → class inherits from class, which in turns inherit from another class



eg:- class Student {
 int reg-no;
 void getNo (int no) {
 reg-no = no;
 }
 void putNo () {
 SOP (" Reg No: " + reg-no);
 }
}

class Marks extends Student {
 float marks;
 void getMarks (float m) {
 marks = m;
 }
 void putMarks () {
 SOP (" Marks " + marks);
 }
}

class Sports extends Marks {
 float score;
 void getScore (float sc) {
 score = sc;
 }
 void putScore () {
 SOP (" Score: " + score);
 }
}

```

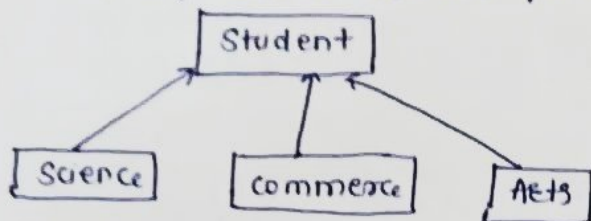
public class Test {
    psvm (String args []) {
        Sports ob = new Sports ();
        ob.getNo (112);
        ob.putNo ();
        ob.getMarks (78);
        ob.putMarks ();
        ob.getScore (68.7);
        ob.putScore ();
    }
}
  
```

volatile

↳ tells JVM that variable may be changed by multiple threads & ensures visibility of changes to variables across threads.

• Hierarchical Inheritance

↳ no of classes derived from one base class.



eg:-

```

class Student {
    public void one() {
        SOP("Student method called");
    }
}
  
```

```

class Science extends Student {
    public void sci() {
        SOP("Sci method called");
    }
}
  
```

```

class Commerce extends Student {
    public void com() {
        SOP("com. method called");
    }
}
  
```

```

class Arts extends Student {
    public void art() {
        SOP("art method called");
    }
}
  
```

```

public class Test {
    public static void main (String args[]) {
        Science ob1 = new Science();
        Commerce ob2 = new Commerce();
        Arts ob3 = new Arts();
        ob1.sci();
        ob2.com();
        ob3.art();
    }
}
  
```

* Hybrid

↓
Combinatⁿ of two or more types of inheritance.

→ possible only through interfaces.

* Multiple Inheritance

↓
class inherits from more than one class

→ does not support in Java.

↓
why?

- ambiguity
- Diamond problem.

* ambiguity

↓
occurs when two or more parent class have methods with same name, child class doesn't know which to use.

eg:-

```

class A {
    void show() {
        SOP("A");
    }
}
  
```

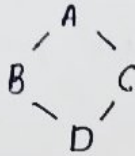
```

class B {
    void show() {
        SOP("B");
    }
}
  
```

X class C extends A, B { }

• Diamond Problem.

↳ Special case of ambiguity.



• class B & C → inherits from A.

• class D → inherits from both B & C

→ To solve this, use interface.

* Polymorphism

allows method/obj. to take multiple forms.

Types

→ Compile time → Method overloading

→ Runtime → Method Overriding.

* Compile time → handle - compiler

↳ Same method name but diff. param.

* Run-time

↳ Two diff. classes have methods with same name & param.

- can't override private, static, final method.

- handle - by JVM.

* Abstract?

↳ hiding implementation details & showing only functionality.

eg. Driving Car → you press the accelerator but you don't know engine's internal working.

Abstract? can be achieved in two ways:-

1] Abstract classes

2] Interfaces.

Abstract classes

→ can have both abstract & concrete methods.

→ support instance var.

→ can have public, private, protected methods

→ partial abstract?

Interfaces.

- only abstract method.

- only support static final variables.

- methods - public by default.

→ 100% abstract?

Functional interface → has only one abstract method.

@FunctionalInterface.

marker interface

↳ interface with no methods / fields.

Key points:-

• main method can't get overloaded.

• static method can also get overloaded.

eg:- Women

Abstract? vs Encapsulation
↳ Focuses on what to expose, hides implementation.

Encapsulation?
↳ Focuses on how data is hidden inside class.

Mobile phone:-

↳ you can tap on icons, but you can't see how OS loads to it.

* Encapsulation

↳ wrapping / binding of data & methods together in a single unit.

→ implementation → using private var & public getter & setters.
data hiding. control

eg. ATM Machine

user → insert card, enter pin, withdraw cash.

Can't see → acc. details, How bal. calculated, How pin is verified.

ATM hides internal data & logic, gives access to what's necessary.

Access Modifiers

↳ accessibility / scope of any var / methods.

- Public → accessible everywhere
- Private → accessible within same class.
- Protected → outside package but only in sub class.
- Default → if no modifier specified, accessible within same package.

Key words:-

• Static

↳ used → memory management.

• belongs to class

• class-level data.

```
class student {
    String name;
    static String school;
}

public class OOPS {
    public static void main(String args[]) {
        student ob = new student();
        ob.name = "darshana";
        ob.sopIn(ob.school);
    }
}
```

Use →
→ variable
→ method
→ block

- variable - common prop. for every obj.
- method - directly access by its class name, no need to create obj.
- block - execute before main method.

execute at the time of class loading.

Note:- we can have multiple static blocks.

• Super

↳ access data of parent class.

- use at obj. level.

access constructor,

var, method of parent class

Final
→ variable
→ method
→ class

- variable - constant
- method - cannot override
- class - cannot extend.

when to use static?

memory bachani ho!

• this

↳ use →
→ variable
→ method
→ constructor.

- variable - refer current class instance var.
- method - used to call another method of same class
- constructor - used to call constructor within same class.
- ↳ useful → constructor chaining.

* Static vs Instance

Static

- memory allocated with class
- directly access by class name
- Common for all obj. val.

Instance

- memory allocated separately for each obj.
- access through obj.
- Value - different for all obj.

Exception Handling → mechanism → handle run time errors.

- Exception:- unexpected event occurs during executⁿ of program., disrupts normal flow.

Types

- checked
- unchecked.

* can main method get overloaded?
yes JVM always calls main method which receives string as an args

- checked → occurs at compile time.
 - must handle using try, catch or throws.

- eg:-
- IOException
 - SQLException
 - FileNotFoundException
 - ClassNotFoundException

Garbage collector: delete unused data

try-with-resources.

↳ automatically close resources.

- JDK 1.7.

- Runtime (Unchecked) → occurs at run time.

- eg:-
- NullPointerException
 - ArithmeticException
 - ArrayIndexOutOfBoundsException.

- calls (or does)
Runtime (unchecked)
↳ programmer की गलती के कारण आती है

* Error

↳ usually caused by system failures, not program.

- eg:-
- OutOfMemoryError - JVM runs out of memory
 - StackOverflowError - due to many method executⁿ invocatⁿ
 - VirtualMachineError

* try, catch, finally

- try - encloses code that may throw exception
- catch - handles exceptⁿ if it occurs.
- finally - always execute even if exceptⁿ occurs.

* throw vs throws

→ only used → compile time exceptⁿ
→ indicates caller method.

↓
handle single exceptⁿ

↓
handles multiple exceptⁿ.

Note:- we can have multiple catch blocks.

- we can use multiple exceptⁿ in single catch blocks by using ' | ' operator since java 7+

Multithreading.

eg:- File downloading

multiple threads runs within same process.

- Multiprocessing
↳ multiple process runs independently.
- multitasking
↳ execute multiple task simultaneously.

ways

- ↳ process based → each process allocates separate memory area.
- ↳ thread based → share same add. space.

* Thread class.

↳ provides constructor & methods to create & perform operation on thread.

— extends obj. class & implement runnable interface.

* Thread life cycle

1. New → born

2. Active

↳ Runnable → ready to run

↳ Running → currently executing

3. Blocked / waiting → paused

4. Terminate → finished execution

* Thread scheduler

↳ decides which thread to run & which thread to execute.

* Thread Priority.

↳ provides some priorities, acc. to these priorities JVM allocates to the processes.

Range → 1 to 10

Types

↳ MAX_PRIORITY = 10

↳ NORM_PRIORITY = 5

↳ MIN_PRIORITY = 1

- If priority is set to be high, it will throw error, i.e. `IllegalArgumentException`.
- To set thread priority, use `setPriority(int priority)`.

methods →

- `public static void setPriority()`
- `public final int getPriority()`

Runnable vs Thread

Interface, allows multiple inheritance

Thread → class, extends directly.

Runnable vs Callable

• Runnable → No return, no exception.

• Callable → returns val, can throw checked exception.

* Thread Control methods

- `sleep()` - pause for specific period
- `join()` - wait until another thread
- `yield()` - stop current executing thread to execute.
- `isAlive()` - checks if thread is still

Synchronisation → prevent race condⁿ, ensure

↳ technique, through which we can threads, among the no. of threads one thread will enter inside the synchronised area.

purpose

↳ overcome the problem of multithreading.

eg. Ticket Booking System

* Lambda exp

↳ shortcut for writing anonymous methods / functⁿ.

Syntax:-

(param) → { code }

* Stream API (Java 8)

↳ way to process collectⁿ in functional style.

operatⁿ

- ↳ `filter()` - filters elements
- `map()` - transform elements
- `sorted()` - sort elements
- `collect()` - collect results to list
- `forEach()` - loop through each item.

Covariant return type

↓
overridden method can return subclass type of original return type.

* File Handling

↳ provides `java.io` & `java.nio` package to read / write files.

* Serialisatⁿ

↳ converting java obj into byte stream

Deserialisatⁿ → converting byte stream back to obj.

Java 8 Features.

- ↳ Lambda exp
- ↳ Stream API
- ↳ Functional Interface
- ↳ Default & Static method in Interf
- ↳ Method Reference
- ↳ Optional class
- ↳ Date & Time API

Synchronized method vs block.

method: locks entire method

Block = locks only specific section.

Volatile

↳ ensures variable is always read from main memory, not CPU cache. (latest val)

Dynamic method dispatch

↓
at runtime, method call is resolved based on object, not ref. type.

Inter-thread Communicatⁿ methods:-

- `wait()` - Makes thread wait
- `notify()` → wakes up one waiting thread
- `notifyAll()` → wakes up all multiple waiting thread.

Strings

↳ seq. sequence of characters.

- methods

- concat()
- equals()
- split()
- length()
- replace()
- compareTo()
- substring()

- Two ways to create string obj.

① String literal

② new keyword

* String, StringBuffer, StringBuilder

String

- Immutable
- Thread-safe
- Slow

StringBuffer

- mutable
- Thread-safe
- Slow

StringBuilder

- mutable
- Not thread-safe
- Fastest

Types of Memory in JVM

- Heap - stores obj.
- Stack - stores method calls & local variables.
- Method area - stores class metadata.
- PC Register area - stores add. of current instruction. (add val to next instruction key)
- Native Method Stack - for non-java code

final, finally, finalize

- final - constant (var)
- finally - block - always executes after try-catch.
- finalize() - called by garbage collector before obj. is reclaimed.

* Autoboxing → primitive → wrapper automatically.

* Unboxing → wrapper → primitive

#

```
try (BufferedReader br = new  
    BufferedReader (new FileReader  
        ("data.txt"))) {  
    sop (br.readLine());  
} catch (IOException e) {  
    e.printStackTrace();  
}
```

⇒ No need to write br.close(), because JVM calls it automatically.

Throw

→ create exceptⁿ object manually.

- used - runtime exceptⁿ
- used - single exceptⁿ
- inside method

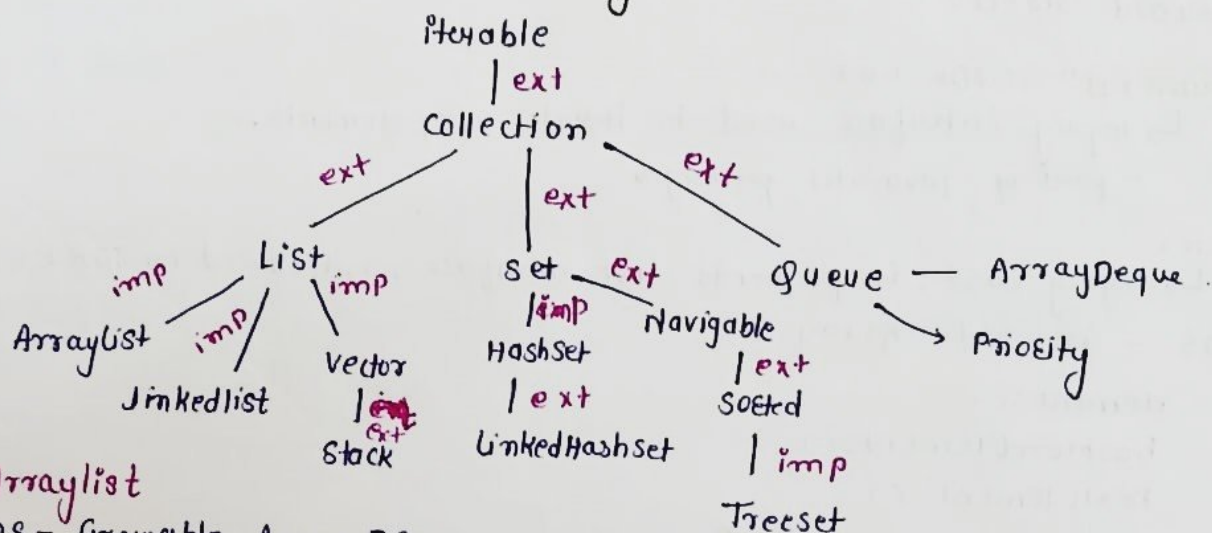
- use with new keyword
- can not write statement after throw keyword.

Collection Framework

Set of classes & interfaces, used to store, retrieve & manipulate data.

- Collection
↳ single entity, used to store obj. type of data.
- Framework
↳ set of provide structured, reusable & extensible approach to handle data collectⁿ

* Collection Framework Hierarchy.



* ArrayList

- DS - Growable Array DS.
- follows insertion order.
- duplicacy allowed.
- store homogeneous type of data.
- store multiple null values.
- indexed based DS.

* LinkedList vs ArrayList

- | | |
|---|--|
| ↓ | ↓ |
| • non-contiguous mem. locat ⁿ | • contiguous memory locat ⁿ |
| • best - insert ⁿ & delet ⁿ | • worst - insert ⁿ & delet ⁿ |
| • worst - searching | • best - searching |
| • DS - Doubly linkedlist | • DS - Growable Array DS. |

Iterable

root interface in JCF, rep. collectⁿ can be iterated.
→ JDK-1.5

Iterator

Interface, displays collectⁿ of obj one-by-one.

- **ListIterator**
- **Enumeratⁿ**

* Iterator vs ListIterator

Iterator

- displays collection of obj one-by-one
- 2 methods
 - hasNext()
 - next()
- print elements only in forward direction

ListIterator

- displays only list implemented class.
- 3 methods
 - hasNext()
 - hasPrevious()
 - previous()
- print ele. in both direction

* Enumerated (JDK 1.0)

↳ legacy interface used to iterate over elements.
- part of java.util packages.

* Vector

↳ legacy class, implements list interface, introduced in JDK 1.0

- DS - Growable Array DS.
- elements()
 - hasMoreElements()
 - nextElements()
- only displays fwd direction
- can iterate vector through enumerated.
- Same like ArrayList.

* Stack

↳ child class of vector, implemented by list interface.

- DS - LIFO
- indexed base
- does not follow sorting order
- duplicacy allowed.
- multiple elements can be stored.

* List vs Set

List

- does not follow sorting order.
- follows insert order.
- duplicate data - allowed
- multiple null values - allowed
- indexed based

Set

- does not follow insert order.
- duplicate data - not allowed.
- only single null val. can be stored.
- does not follow index based.

* HashSet

↳ not indexed based ds.

- according to hashCode val, hashset store elements.
- does not follow insertⁿ order.
- can store single null values.
- back up by Map.

* HashSet vs LinkedHashSet

HashSet

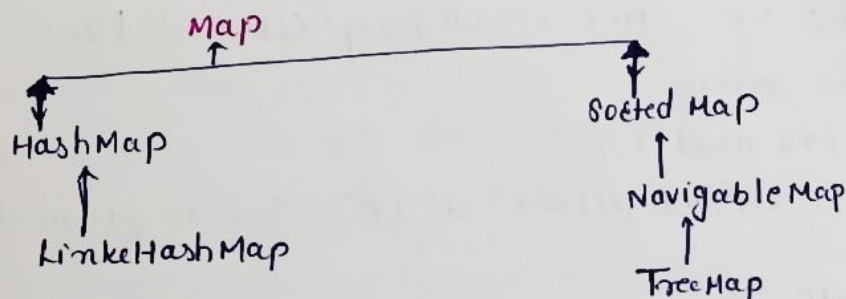
- implemented class of Set interface.
- DS - Hash table
- does not follow insertⁿ order.

LinkedHashSet

- child class of HashSet, implements Set interface.
- DS - Hash table + Doubly Linked list.
- follow insertⁿ order.

* TreeSet

- ↓
- implemented class of Navigable Set, sorted Set & Set interface.
- DS - balanced tree
- follows sorting order.
- does not follow insertⁿ order.
- Contain unique elements.
- null elements - not allowed.



* HashMap

↳ implemented class of Map interface.

- DS - Hash table
- does not follow insertⁿ order
- acc. to hashCode values, hashmap store elements.
- duplicate keys - not allowed, but can store duplicate val.
- Non-synchronised.
- If we want to print key & values separately, so we use entry set.
entry set - entry interfaces.
↳ returns value inside the set
- can retrieve data separately in HashMap.

* TreeMap



implemented class of navigable Map, Sorted Map & Map interface.

- DS - Red Black Tree.
- store homogeneous type of data.
- Data retrieval - sorting orders
- null key - not allowed.
- values could be duplicate.

JDBC → Java Database Connectivity

↳ API, enables java app. to interact with DB.

Steps:-

1. Load JDBC Driver.

```
class.forName("com.mysql.cj.jdbc.Driver");
```

2. Establish Connection

```
Connection con = DriverManager.getConnection("jdbc:mysql://localhost:3306/my-db", "root", "pass");
```

3. Create Statement

```
Statement stmt = con.createStatement();
```

4. Execute Query

```
ResultSet rs = stmt.executeQuery("select * from users");
```

5. Process the result

```
while (rs.next()) {  
    SOP(rs.getInt("id") + " " + rs.getName("name"));  
}
```

6. Close Connection

```
rs.close();  
stmt.close();  
con.close();
```

- DriverManager - establish connection to db using getConnection().
- Connection - Interface, rep session b/w Java app & db
- Statement - sends static SQL queries to db
- ResultSet - holds data returned by select query.
- executeQuery() → SELECT, returns ResultSet.
- executeUpdate() → Insert, update, delete.

* Types of JDBC Drivers

- ↳ Type 1 → JDBC - ODBC bridge
- Type 2 → Native API
- Type 3 → Native Protocol
- Type 4 → Thin Driver.

- Serializatiⁿ - convert obj. to byte stream
- Deserializatiⁿ - byte stream to obj.

* transient vs volatile
↳ ignore var during serializatiⁿ

ensures visibility of changes across threads.

* SQL injectⁿ

- ↳ security issue where attackers injects SQL code.
- avoid it using prepared statement.



execute parametrized queries.

* Statement, prepared statement, Callable statement.

- Statement: for executing static SQL queries.
- Prepared statement: for dynamic SQL queries.
- Callable statement: To call stored procedures from db.

* Servlet → server side technology.

- ↳ java class - used to build web app.

- runs on Java enabled server like Apache Tomcat & handles HTTP request & sends responses.

* Life Cycle.

- init() - called once when servlet is created.
- service() - called every time a request comes in.
- destroy() - called once when the servlet is being destroyed.

* Classes

- HttpServlet - Base class for HTTP servlets.
- HttpServletRequest - Rep. HTTP req. from client.
- HttpServletResponse - Rep. HTTP resp. to be sent.
- ServletConfig - For servlet configuratiⁿ.
- ServletContext - For app. wide parameters.

* Methods.

- doGet() - handles HTTP GET req.
- doPost() - handles HTTP POST req.
- init() - initializes servlet.
- destroy() - called when servlet is being removed.

doGet()

query in

→ data sent in URL string

→ less secure

→ use - reading / retrieving data

doPost()

→ data sent in response body

→ more secure

→ submit sensitive data

JSP

↳ stands for Java Server Pages.

- used to create dynamic HTML pages using Java code embedded inside HTML.

* Tags.

- Scriptlet (`<% code %>`) - java code inside Jsp
- Expression (`<%= exp %>`) - o/p results of Java exp.
- Declaration (`<%! type var = val; %>`) - Declares var/methods.
- Directive (`<% @ directive %>`) - Controls page behavior.

Spring Boot

↳ Rapid App. Development.

↳ framework

- Submodule of spring to build web app.
- Develop & maintained by "Pivotal Software".

↓
enterprise slow comp.

Why Spring Boot?

- Auto configuration.
 - Embedded server.
 - No xml configuration.
 - Easy dependency management via starters.
- * `@SpringBootApplication` → marks the main class of Spring Boot App.

↓
Combination of `@Configuration`

`@EnableAutoConfiguration`

`@ComponentScan`.

* auto-configuration

↓
automatically configures beans based on dependencies using `@EnableAutoConfiguration`.

* Spring Boot starters.

↓
predefined set of dependencies for specific purpose.

eg:-

spring-boot-starter-web

spring-boot-starter-data-jpa.

spring-boot-starter-security.

* Spring vs Spring Boot

- Manual XML config.
- Server - req. deployment

* Default embedded Server - Tomcat

* Default port - 8081

* purpose → application.properties → server settings, DB config.

* Spring Boot Actuator

↳ exposes product ready end-points health, metrics, beans etc.
eg. /actuator/health, /actuator/info

* To connect DB in Spring Boot.

↳ Add JPA / JDBC dependency.

→ set DB prop. in application.properties

→ create repo using JpaRepository.

• JPA

↳ Java Persistence API - 2006

→ specificatⁿ, simplifies interactⁿ b/w

→ 2019 → Jakarta Persistence API

↓
provides 2 interfaces

1) CRUDRepository.

2) JpaRepository.

↳ .save

• delete

• update

• get

• API

↳ Application Programming Interface.

→ Bridge b/w system or devices.

eg. Login API

Payment Gateway API

Types

↳ • Public

• Partner

• Private

Spring Boot

• auto-configurⁿ

• embedded servers

health, metrics, beans etc.
• **Spring Framework**.
↓
open source, lightweight framework for building enterprise app.

↳ Benefits:-

• loose coupling b/w comp.

• Testability

• Modularity

• Easy integratⁿ with ORM.

Java obj. & Relational DB.

• **DI** - Dependency Injectⁿ.
- design pattern, one obj. injects into another.

Types

↳ Constructor-based

↳ Setter-based

↳ Field injectⁿ.

• **BeanFactory vs ApplicationContext**

modules of Spring.

- Core

- Beans

- Context

- AOP

- Spring MVC

- Spring JDBC

- * Open API - can be used by anyone.
- * Partner API - for business to business (B2B)
eg. MakeMyTrip.
- * Private API - only for specific or internal working
- * Composite API - combines any two or more API's for any System / project.

Web Services

↳ specific type of API designed to follow set of protocols for communication.

Types

- ↳ SOAP → simple object Access Protocol.
- ↳ REST → Representational State Transfer.

* Rest API

↳ architectural style for building web services using HTTP methods. like Get, Post, PUT, DELETE

* Annotations

- @RestController - Marks class as REST controller.
- @RequestMapping - Maps HTTP req. to handler methods.
- @GetMapping - Handles Get req., fetch data.
- @PostMapping - Handles Post req., insert data.
- @PutMapping - update the data.
- @DeleteMapping - Handles Delete req.
- @PathVariable - Extract values from URL.
- @RequestParam - extracts query param. from URL.
- @RequestBody - Maps req. body to Java obj.
- @ResponseBody - Returns java obj. as JSON/XML.

* RestController

↳ @Controller + @ResponseBody
- returns data directly from REST endpoints.

* @Controller vs @RestController.

@Controller

@RestController

→ use - web App.

→ use - Rest APIs.

→ returns views like HTML

→ returns data like JSON / XML.

→ combined with @ResponseBody manually

→ internally includes @ResponseBody automatically.

→ Suitable - MVC apps

→ suitable - Restful services.

* @RequestBody

↳ used to bind HTTP request body to java obj. especially for POST / PUT

* To handle Exceptions → @ControllerAdvice & @ExceptionHandler.

* ResponseEntity

↳ wrapper for HTTP response, allows control over status code, headers & body.

* @Autowired

↳ injects bean automatically into dependent class.

* @Component

@Service

@Repository

@Controller

↓
Generic bean

↓
Business Logic layer.

↓
DAO layer.

↓
MVC Controller

* @Value

↳ used to inject values from application.properties.

* Spring Boot Dev Tools. - enables

↳ auto-restart on code changes
- live reload.

* Spring Boot Initializer

- Developer productivity tools.

web tool → generate Spring Boot projects with req dependencies.

Spring Boot starters :- pre configured dependencies.

To handle transact? → use @Transactional.

Spring Beans:-

↳ obj, managed by IOC container.

→ responsible for creating bean, injecting dependencies.
Managing life cycle, Destroying it when no longer needed.

* @Controller vs @RestController.

@Controller

@RestController

→ use - web App.

→ use - Rest APIs.

→ returns views like HTML

→ returns data like JSON / XML.

→ combined with @ResponseBody manually

→ internally includes @ResponseBody automatically.

→ Suitable - MVC apps

→ suitable - Restful services.

* @RequestBody

→ used to bind HTTP request body to java obj. especially for POST / PUT

* To handle Exceptions → @ControllerAdvice & @ExceptionHandler.

* ResponseEntity

→ wrapper for HTTP response, allows control over status code, headers & body.

* @Autowired

→ injects bean automatically into dependent class

* @Component

@Service

@Repository

@Controller

↓
Generic bean

↓
Business Logic layer.

↓
DAO layer.

↓
MVC Controller

* @Value

→ used to inject values from application.properties.

* Spring Boot Dev Tools. — enables

→ auto-restart on code changes
— live reload.

* Spring Boot Initializer

— developer productivity tools.

web tool → generate Spring Boot projects with req dependencies.

Spring Boot starters :- pre configured dependencies.

To handle transactⁿ → use @Transactional.

Spring Beans:-

→ obj, managed by IOC container.

→ responsible for creating bean, injecting dependencies, managing life cycle, destroying it when no longer needed.

Constructor vs Setter Inject?

↓ uses final for field

- Dependencies - provided through class constructor.
- Ensures bean is fully initialized when it's created.

Setter

→ Dependencies are set via public setter methods after obj. is created.

- can reconfigure dependencies after obj. created.
- use for optional dependencies.

Annotation

→ special form of metadata, provides info to the compiler & Spring at runtime.

- used to configure beans, enable features, & reduce XML configuratn.

Types

→ stereotype (Marked classes as Spring beans)

- . @Component
- . @Service
- . @Repository
- . @Controller

→ DI annotation

→ . @Autowired

• @Qualifier → used with @Autowired to inject by name.

• @Primary → Marks bean as a default choice when multiple beans match.

→ Configuratiⁿ ann.

- @Configuratiⁿ
- @Bean
- @ComponentScan

→ Web & Rest.

- @RestController
- @RequestMapping
- @GetMapping, @PostMapping
- @RequestBody
- @ResponseBody

Autowiring

↓
automatically injects bean into another bean w/o explicitly specifying bean in configuratiⁿ modes

- byName
- byType
- constructor
- autodetected

Microservices.

Beans can be configured in 3 main ways.

- ① XML config?
- ② Java-based
- ③ Annotation-based. → @Component

Bean scopes.

- Singleton: one instance per Spring container.
- Prototype: new instance every request.
- Request: one instance per HTTP request.
- Session: one instance per HTTP session.

AOP → Aspect Oriented Programming.

↳ AOP lets you separate cross-cutting concerns from main business logic.

↓.

Real life use:-

- logging
- security checks
- Transaction Management.
- Performance monitoring
- caching.

Code that appears in multiple places.

Bean Life Cycle

1. Instantiated - object created.
2. DI - dependencies injected.
3. Initialization - custom init method runs.
4. Ready to use - Bean available in app. context.
5. Destruction - Destroy method call before bean is removed.

Bean Wiring

↳ Connecting beans together in Spring container so they can work with each other.

- part of DI.

Types

- ↳ explicit (XML)
- ↳ Autowiring (@Autowired)

IoC Container.

↳ core part of framework.

- creates objects
- Manage their life cycle
- Injects dependencies.
- Configures them based on metadata.

Types

- ↳ BeanFactory
- ↳ ApplicationContext.

BeanFactory

↓

- Basic container
- less commonly used directly.

ApplicationContext

↓

- extends BeanFactory
- Adds AOP support, event handling, etc.